

TRANSFORMER - BASED TEXT CLASSIFICATION MODEL

INTRODUCTION:

This document describes a system designed to continuously train and deploy a transformer-based text classification model. The goal is to keep the model up-to-date and effective by automatically retraining it when necessary. This pipeline supports both live and batch inference and is currently set up for local deployment. The document outlines how to monitor model performance, the infrastructure for deployment, and the methods used for inference and cost optimization.

MONITORING MODEL STALENESS:

Monitoring Approach:

To ensure that the model remains effective over time, we need to monitor its performance and check for any changes in the data it processes. This helps to identify when the model might be becoming outdated or less effective.

Key Metrics:

1. **Accuracy Monitoring:** We track the model's accuracy and compare it against a predefined threshold (e.g., 75%). If the accuracy drops below this threshold, it indicates that the model may need retraining.
2. **Data Drift:** We also monitor for data drift, which occurs when the distribution of input data changes over time. This is important because even if the model is accurate initially, changes in the data can reduce its effectiveness.

Tools Used:

- **MLflow:** This tool helps track model performance metrics, including accuracy. It logs these metrics so that we can review them over time.
- **Alert System:** An email alert system is set up to notify us when performance metrics drop below acceptable levels. This can trigger manual or automated retraining.

Monitoring Workflow:

1. **Initial Training:** Train the model and record its initial performance.
2. **Ongoing Monitoring:** Regularly check the model's performance metrics.
3. **Retraining Trigger:** If performance drops or data drift is detected, initiate retraining.

INFRASTRUCTURE FOR DEPLOYMENT:

Local Deployment:

The system is designed to be deployed on a local machine. This setup is simple and cost-effective, though it can be adapted for cloud deployment if needed.

Components:

1. **Model Storage:** The trained model and its configuration are stored locally. This includes the model weights and configuration files.
2. **Live Inference:** FastAPI is used to serve the model through a REST API. This allows the model to make real-time predictions. Uvicorn is used to run the FastAPI server, which listens for incoming requests and provides predictions.
3. **Batch Inference:** A separate script handles batch processing. It reads input data from a CSV file, makes predictions, and writes the results to an output CSV file.
4. **Monitoring and Retraining:** A monitoring script runs periodically to check model performance. If performance metrics fall below the set thresholds, the script triggers retraining.

Local Deployment Summary:

- **FastAPI:** For real-time predictions.
- **Batch Processing:** For handling large datasets efficiently.
- **Monitoring:** For continuous performance evaluation and retraining.

Cloud Deployment (Optional):

If the system needs to be scaled or moved to the cloud, it can be adapted with minimal changes. Cloud services like AWS or GCP can be used to handle live inference and batch processing, offering additional scalability and flexibility.

INFERENCE METHODS AND COST OPTIMIZATIONS:

Inference Methods:

1. **Live Inference** (Real-Time API): The FastAPI server provides a `/predict` endpoint for making real-time predictions. Users send text data, and the server returns sentiment analysis results.
2. **Batch Inference** (CSV Processing): A script processes text data from CSV files in batches. This is useful for analyzing large volumes of data, such as customer feedback or survey responses.

Cost Optimizations:

1. **Model Efficiency**: We use DistilBERT, a smaller and faster version of BERT. It offers good performance while being more resource-efficient.
2. **Batch Size Optimization**: The system processes data in batches, balancing processing speed and resource usage to keep costs down.
3. **Hardware Utilization**: The model leverages GPU acceleration (if available) for faster training and inference. If a GPU is not available, the system falls back to CPU, which is adequate for smaller tasks.
4. **Retraining Efficiency**: Retraining is only triggered when necessary, based on performance metrics or data drift. This avoids unnecessary computations and reduces costs.

Inference and Cost Management:

- **Live Inference**: Handles real-time requests efficiently with FastAPI.
- **Batch Inference**: Processes large datasets with optimized batch sizes.
- **Cost Efficiency**: Uses DistilBERT and optimized hardware usage to manage costs effectively.